SCS 2312 - Computational Models and Programming Language Concepts

Take-home Assignment Report

Student Name: [Your Name Here]
Student ID: [Your ID Here]
Date: February 9, 2026
University: University of Colombo School of Computing

Table of Contents

---

1. Context-Free Grammar (CFG)

The following Context-Free Grammar (CFG) accurately represents the structure of the programming language implemented in this assignment:

1.1 Grammar Rules

```
<Program> ::= <StatementList>

<StatementList> ::= <Statement> <StatementList>
                  | <Statement>

<Statement> ::= <Declaration>
              | <Assignment>
              | <PrintStatement>

<Declaration> ::= "int" <Identifier> ";"
                | "int" <Identifier> "=" <Expression> ";"

<Assignment> ::= <Identifier> "=" <Expression> ";"

<PrintStatement> ::= "print" "(" <Identifier> ")" ";"

<Expression> ::= <Term> "+" <Expression>
```

```
                | <Term>

<Term> ::= <Number>
         | <Identifier>

<Identifier> ::= <Letter> | <Identifier> <Letter> | <Identifier> <Digit> |
<Identifier> "_"

<Number> ::= <Digit> | <Number> <Digit>

<Letter> ::= "a" | "b" | ... | "z" | "A" | "B" | ... | "Z" | "_"

<Digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

1.2 Grammar Explanation

- **Program**: A program consists of one or more statements.
- **StatementList**: A recursive structure allowing multiple statements.
- **Statement**: Can be a variable declaration, assignment, or print statement.
- **Declaration**: Declares an integer variable with optional initialization.
- **Assignment**: Assigns a value (from an expression) to an existing variable.
- **PrintStatement**: Outputs the value of a variable.
- **Expression**: Supports addition operations and can be recursive.
- **Term**: The basic unit of an expression (number or identifier).

1.3 Parse Tree Example

For the input: `int x = 5;`

```
        <Program>
            |
      <StatementList>
            |
       <Statement>
            |
      <Declaration>
       /    |    \
     int    x    =  <Expression>
                        |
                     <Term>
                        |
                    <Number>
                        |
                        5
```

---

2. Tokenizer Implementation

2.1 Token Types

The tokenizer identifies the following token types:

Token Type	Description	Examples
KEYWORD	Reserved words	`int`, `print`
IDENTIFIER	Variable names	`x`, `y`, `myVar`
OPERATOR	Arithmetic & assignment	`+`, `=`
SYMBOL	Delimiters & punctuation	`(`, `)`, `;`
NUMBER	Integer literals	`5`, `20`, `123`

2.2 Tokenization Algorithm

The tokenizer uses a character-by-character scanning approach:

1. **Skip whitespace**: Ignore spaces, tabs, and newlines
2. **Identify alphabetic characters**:
   - Accumulate characters to form keywords or identifiers
   - Check against keyword list
3. **Identify digits**: Accumulate to form number tokens
4. **Identify operators and symbols**: Single-character recognition

2.3 Key Functions

```
int isKeyword(char *str)     // Checks if token is a keyword
int isOperator(char *str)    // Checks if token is an operator
int isSymbol(char *str)      // Checks if token is a symbol
int isIdentifier(char *str)  // Validates identifier rules
int isNumber(char *str)      // Validates numeric format
```

2.4 Tokenizer Output Example

Input: `int y = 5;`

Output:

```
Token 0: Type = KEYWORD, Value = int
Token 1: Type = IDENTIFIER, Value = y
Token 2: Type = OPERATOR, Value = =
Token 3: Type = NUMBER, Value = 5
Token 4: Type = SYMBOL, Value = ;
```

---

3. Parser Logic Implementation

3.1 Recursive Descent Parsing

The parser implements a **recursive descent** approach, where each non-terminal in the grammar has a corresponding parsing function:

Grammar Rule	Function
`<Program>`	`parseProgram()`
`<StatementList>`	`parseStatementList()`
`<Statement>`	`parseStatement()`
`<Declaration>`	`parseDeclaration()`
`<Assignment>`	`parseAssign()`
`<PrintStatement>`	`parsePrint()`
`<Expression>`	`parseExpression()`

3.2 Parser Functions Explanation

3.2.1 `parseProgram()`

- Entry point for parsing
- Calls `parseStatementList()`
- Checks for unexpected tokens at the end

3.2.2 `parseStatementList()`

- Iterates through all statements in the program
- Calls `parseStatement()` for each statement

3.2.3 `parseStatement()`

- Determines statement type by examining current token
- Routes to appropriate parsing function:
  - `int` keyword → `parseDeclaration()`
  - Identifier → `parseAssign()`
  - `print` keyword → `parsePrint()`

3.2.4 `parseDeclaration()`

- Expects: `int` [=] `;`
- Adds variable to symbol table
- Handles both initialized and uninitialized declarations

3.2.5 `parseAssign()`

- Expects: `= ;`
- Updates variable value in symbol table
- Checks if variable exists

3.2.6 `parsePrint()`

- Expects: `print ( ) ;`
- Retrieves and outputs variable value

3.2.7 `parseExpression()`

- Handles arithmetic expressions with `+` operator
- Returns the computed integer value
- Supports recursive addition: `a + b + c`

3.3 Helper Function: `expect()`

```
void expect(char *type, char *value)
```

This function:

- Verifies the current token matches expected type and value
- Advances to the next token
- Reports syntax error if mismatch occurs

---

4. Error Detection (Syntax & Semantic)

4.1 Syntax Error Detection

The parser detects syntax errors including:

Error Type	Example	Detection Method
Unexpected token type	`int 123;`	Type mismatch in `expect()`
Missing semicolon	`int x = 5`	Expected `;` not found
Invalid statement	`xyz 5;`	No matching statement pattern
Unrecognized token	`int x @ 5;`	Character `@` not valid
Missing parentheses	`print x;`	Expected `(` or `)` not found

Example:

```
if (strcmp(tokens[currentTokenIndex].value, value) != 0)
{
    printf("Syntax Error: Expected '%s' but found '%s'\n",
           value, tokens[currentTokenIndex].value);
    exit(1);
}
```

4.2 Semantic Error Detection

The parser detects semantic errors including:

Error Type	Example	Detection Method
Variable redeclaration	`int x = 5; int x = 10;`	Check in `addVariable()`
Undefined variable	`print(z);` when z not declared	Check in `lookupVariable()`
Uninitialized variable	`int x; int y = x;`	Check `isInitialized` flag
Case sensitivity	`int x = 5; print(X);`	Exact string matching

Example - Redeclaration Check:

```
for (int i = 0; i < variableCount; i++)
{
    if (strcmp(variables[i].name, name) == 0)
    {
        printf("Error: Variable '%s' already declared\n", name);
        exit(1);
    }
}
```

Example - Uninitialized Variable Check:

```
if (variables[i].isInitialized)
{
    return variables[i].value;
}
else
{
    printf("Error: Variable '%s' is not initialized\n", name);
    exit(1);
}
```

4.3 Symbol Table

The symbol table is implemented as an array of `Variable` structures:

```
typedef struct
{
    char name[VARIABLE_NAME_LENGTH];
    int value;
    int isInitialized;
} Variable;
```

Functions:

- `addVariable()`: Add new variable with initialization status
- `setVariableValue()`: Update existing variable value
- `lookupVariable()`: Retrieve variable value (with checks)

---

5. Arithmetic Operations & Execution

5.1 Expression Evaluation

The `parseExpression()` function evaluates arithmetic expressions:

1. **Parse left operand** (number or identifier)
2. **Check for operator** (`+`)
3. **Recursively parse right side**
4. **Return computed result**

5.2 Supported Operations

- **Addition** (`+`): Binary operator supporting multiple operands
- **Variable lookup**: Retrieves value from symbol table
- **Literal numbers**: Direct integer values

5.3 Evaluation Example

Input: `int z = x + y;` where `x = 20` and `y = 5`

Evaluation Steps:

1. Parse `x` → lookup → returns `20`
2. Detect `+` operator
3. Recursively parse `y` → lookup → returns `5`
4. Compute: `20 + 5 = 25`
5. Store result in variable `z`

5.4 Output Generation

The `parsePrint()` function:

1. Validates syntax: `print(identifier);`
2. Looks up variable value
3. Outputs the integer value using `printf("%d\n", value);`

---

6. Code Explanation

6.1 Data Structures

Token Structure

```
typedef struct
{
    char type[TOKEN_TYPE_LENGTH];
    char value[TOKEN_VALUE_LENGTH];
} Token;
```

Stores each token's type and value.

Variable Structure

```
typedef struct
{
    char name[VARIABLE_NAME_LENGTH];
    int value;
    int isInitialized;
} Variable;
```

Maintains the symbol table with initialization tracking.

6.2 Global Variables

```
Token tokens[NUMBER_OF_TOKENS];      // Array of tokens
int currentTokenIndex = 0;           // Current position in token array
int tokenCount = 0;                  // Total tokens parsed

Variable variables[MAX_VARIABLES];   // Symbol table
int variableCount = 0;               // Number of variables
```

6.3 Program Flow

1. **File Reading**: Open input file from command-line argument
2. **Tokenization**: Convert source code to tokens
3. **Token Display**: Print all identified tokens
4. **Parsing**: Validate syntax following CFG rules
5. **Execution**: Evaluate expressions and execute print statements
6. **Cleanup**: Close file and exit

6.4 Key Design Decisions

- **Single-pass parser**: Tokens are consumed sequentially
- **Symbol table**: Array-based for simplicity
- **Error handling**: Immediate exit on error with descriptive message
- **Initialization tracking**: Prevents use of uninitialized variables
- **Case-sensitive identifiers**: Enforces strict naming

---

7. Testing & Results

7.1 Test Case 1: Basic Program (Given Example)

Input File (input.txt):

```
int y = 5;
int x = 20;
int z = x + y;
print(z);
```

Expected Output:

```
25
```

Actual Output:

```
Token 0: Type = KEYWORD, Value = int
Token 1: Type = IDENTIFIER, Value = y
Token 2: Type = OPERATOR, Value = =
Token 3: Type = NUMBER, Value = 5
Token 4: Type = SYMBOL, Value = ;
Token 5: Type = KEYWORD, Value = int
Token 6: Type = IDENTIFIER, Value = x
Token 7: Type = OPERATOR, Value = =
Token 8: Type = NUMBER, Value = 20
Token 9: Type = SYMBOL, Value = ;
Token 10: Type = KEYWORD, Value = int
Token 11: Type = IDENTIFIER, Value = z
Token 12: Type = OPERATOR, Value = =
Token 13: Type = IDENTIFIER, Value = x
Token 14: Type = OPERATOR, Value = +
Token 15: Type = IDENTIFIER, Value = y
Token 16: Type = SYMBOL, Value = ;
Token 17: Type = KEYWORD, Value = print
Token 18: Type = SYMBOL, Value = (
Token 19: Type = IDENTIFIER, Value = z
Token 20: Type = SYMBOL, Value = )
Token 21: Type = SYMBOL, Value = ;
25
```

Result: ✅ PASS

7.2 Test Case 2: Uninitialized Variable

Input:

```
int x;
int y = x + 5;
```

Expected Output:

```
Error: Variable 'x' is not initialized
```

Result: ✅ PASS (Semantic error detected)

7.3 Test Case 3: Undefined Variable

Input:

```
int x = 5;
print(y);
```

Expected Output:

```
Error: Variable 'y' not found
```

Result: ✅ PASS (Semantic error detected)

7.4 Test Case 4: Variable Redeclaration

Input:

```
int x = 5;
int x = 10;
```

Expected Output:

```
Error: Variable 'x' already declared
```

Result: ✅ PASS (Semantic error detected)

7.5 Test Case 5: Syntax Error - Missing Semicolon

Input:

```

```

```
int x = 5
print(x);
```

Expected Output:

```
Syntax Error: Expected ';' but found 'print'
```

Result: ✅ PASS (Syntax error detected)

7.6 Test Case 6: Multiple Additions

Input:

```
int a = 5;
int b = 10;
int c = 15;
int result = a + b + c;
print(result);
```

Expected Output:

```
30
```

Result: ✅ PASS

7.7 Test Case 7: Case Sensitivity

Input:

```
int x = 5;
print(X);
```

Expected Output:

```
Error: Variable 'X' not found
```

Result: ✅ PASS (Case sensitivity enforced)

---

8. Conclusion

8.1 Implementation Summary

This assignment successfully implements:

1. ✅ **Context-Free Grammar (15 marks)**: A complete CFG with clear production rules
2. ✅ **Tokenizer (20 marks)**: Character-by-character scanning with all token types
3. ✅ **Parser Logic (30 marks)**: Recursive descent parser following CFG rules
4. ✅ **Error Detection (20 marks)**: Both syntax and semantic error handling
5. ✅ **Arithmetic & Execution (15 marks)**: Expression evaluation and output

8.2 Features Implemented

- **Comprehensive tokenization** with proper categorization
- **Recursive descent parsing** aligned with CFG
- **Symbol table management** with initialization tracking
- **Syntax error detection** with helpful messages
- **Semantic error detection** (redeclaration, undefined, uninitialized)
- **Expression evaluation** with addition operator
- **Print statement execution** with variable lookup
- **Case-sensitive variable names**

8.3 Compilation & Execution

Compilation:

```
gcc parser.c -o parser
```

Execution:

```
./parser input.txt
```

8.4 Code Quality

- **Error-free compilation**: No warnings or errors
- **Clean code structure**: Modular functions with clear purpose
- **Proper memory handling**: Fixed-size arrays, no dynamic allocation issues
- **Comprehensive error messages**: User-friendly error reporting
- **Well-documented**: Clear function names and logical flow

Appendix A: Complete Source Code

See attached parser.c file

Appendix B: Grammar Notation

- `::=` means "is defined as"
- `|` means "or" (alternatives)
- `<>` denotes non-terminals
- `""` denotes terminals (literal symbols)

End of Report